LangGraph Components

Dr. Sunil Saumya

Dept of Data Science & Artificial Intelligence
Indian Institute of Information Technology Dharwad

sunilsaumya @iiitdwd.ac.in



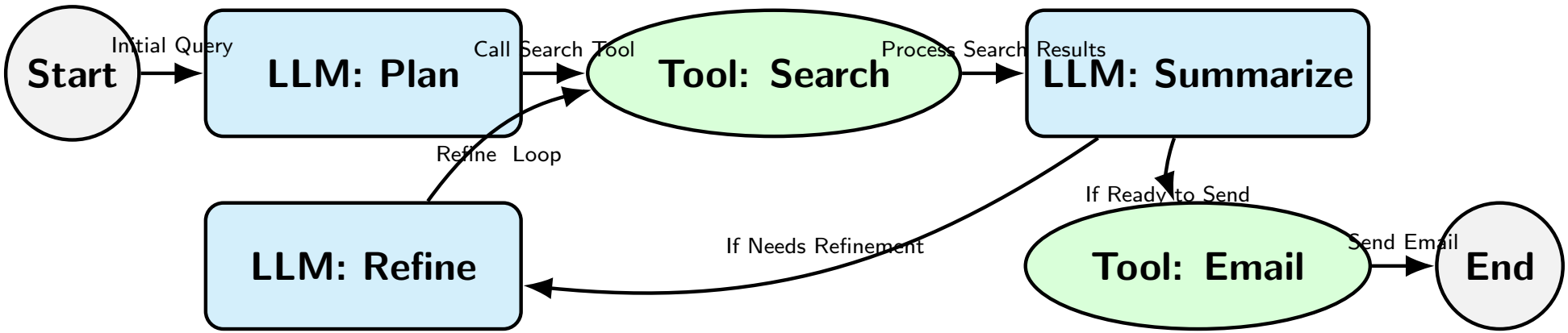
August 13, 2026

Outline

1. LangGraph Definition
2. LangGraph Components
3. LangGraph Execution Model

What is LangGraph?

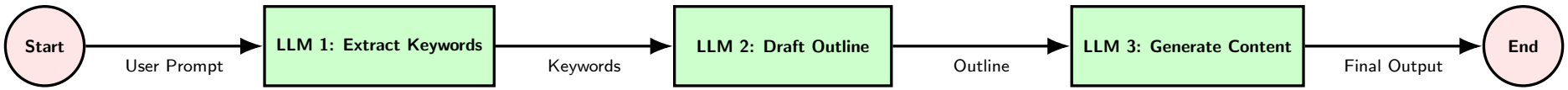
- **Definition:** LangGraph is an orchestration framework for building intelligent, **stateful**, and **multi-step LLM workflows**.
- **Purpose:** It's designed for **agentic** and production-grade AI applications by enabling advanced features like:
  - **Parallelism** (running multiple steps simultaneously)
  - **Loops** and **branching** (conditional logic)
  - **Memory** and **resumability** (retaining state across steps and sessions)
- **Modeling:** It models your entire application logic as a **graph** structure:
  - **Nodes** represent tasks or actions (e.g., call LLM, use tool).
  - **Edges** represent routing or the flow between tasks.



What are LLM Workflows?

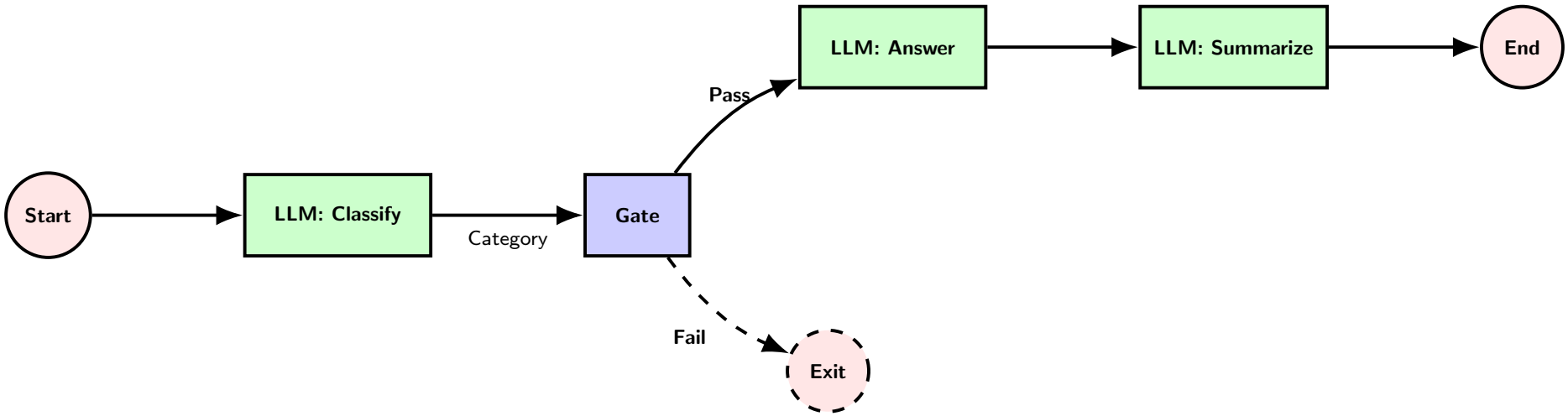
- **LLM workflows** are a step-by-step process using which we can build complex LLM applications.
- Each step in a workflow performs a **distinct task** - such as prompting, reasoning, tool calling, memory access, or decision-making.
- Workflows can be **linear, parallel, branched, or looped**, allowing for complex behaviours like:
  - Retries (handling failures)
  - Multi-agent communication
  - Tool-augmented reasoning
- **Common workflows**

LLM Workflow: Sequential (Prompt Chaining)



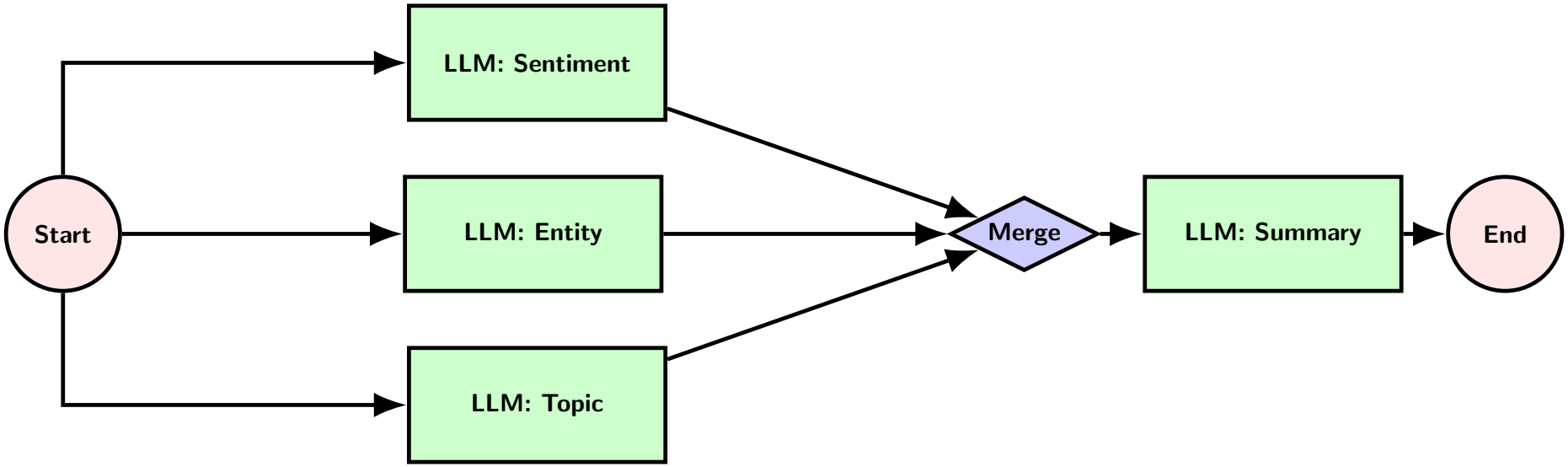
- Tasks are performed **in sequence**, with each LLM output feeding the next prompt.
- Commonly used for multi-step reasoning or structured text generation.
- Enables refinement across stages — from idea extraction to content synthesis.
- Simple yet effective workflow for deterministic pipelines.

LLM workflow: Query Routing



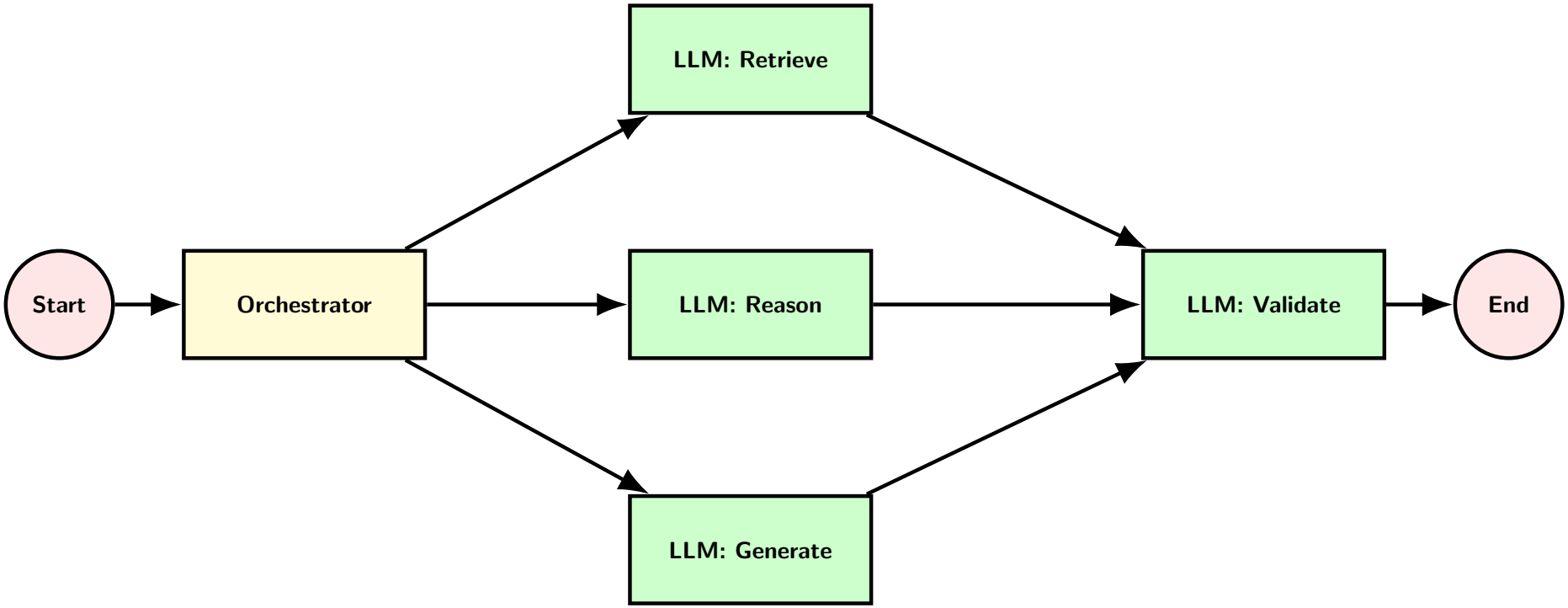
- **LLM 1** classifies the user query (e.g., Factual vs. Ambiguous).
- The **Gate** routes the workflow based on the classification.
- If Factual (**Pass**), the query proceeds to the answer generator.
- If Ambiguous (**Fail**), the process stops for human review.

LLM Workflow: Parallelization



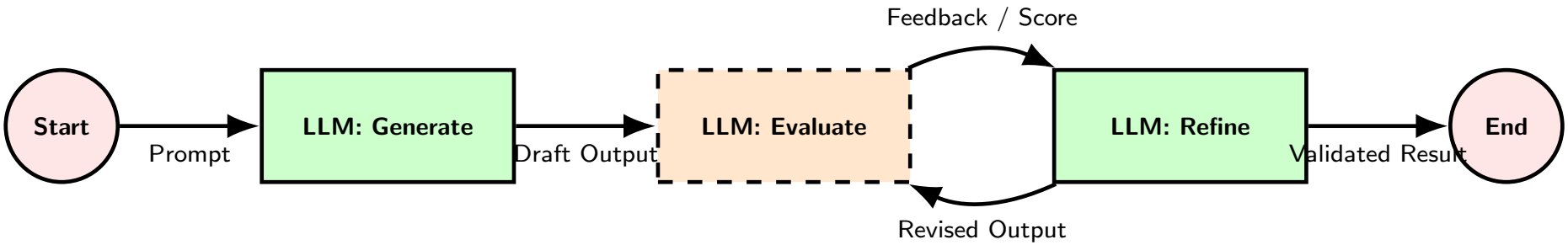
- Multiple **LLM modules** run in parallel for faster processing (e.g., sentiment, topic, and entity extraction).
- The **Merge node** aggregates parallel outputs.
- Final **LLM** synthesizes results into a unified response.

LLM Workflow: Orchestrator Pattern



- The **Orchestrator** coordinates multiple LLM sub-tasks.
- Each sub-LLM handles a specific function (Retrieve, Reason, Generate).
- Final **Validation LLM** ensures coherence and correctness.
- Enables modular, dynamic workflow execution controlled by one manager node.

LLM Workflow: Evaluator Pattern



- **Evaluator LLM** reviews and scores the initial generation.
- **Refiner LLM** iteratively improves output based on feedback.
- The cycle continues until quality thresholds are met.
- Useful for self-correcting and reinforcement-style workflows.

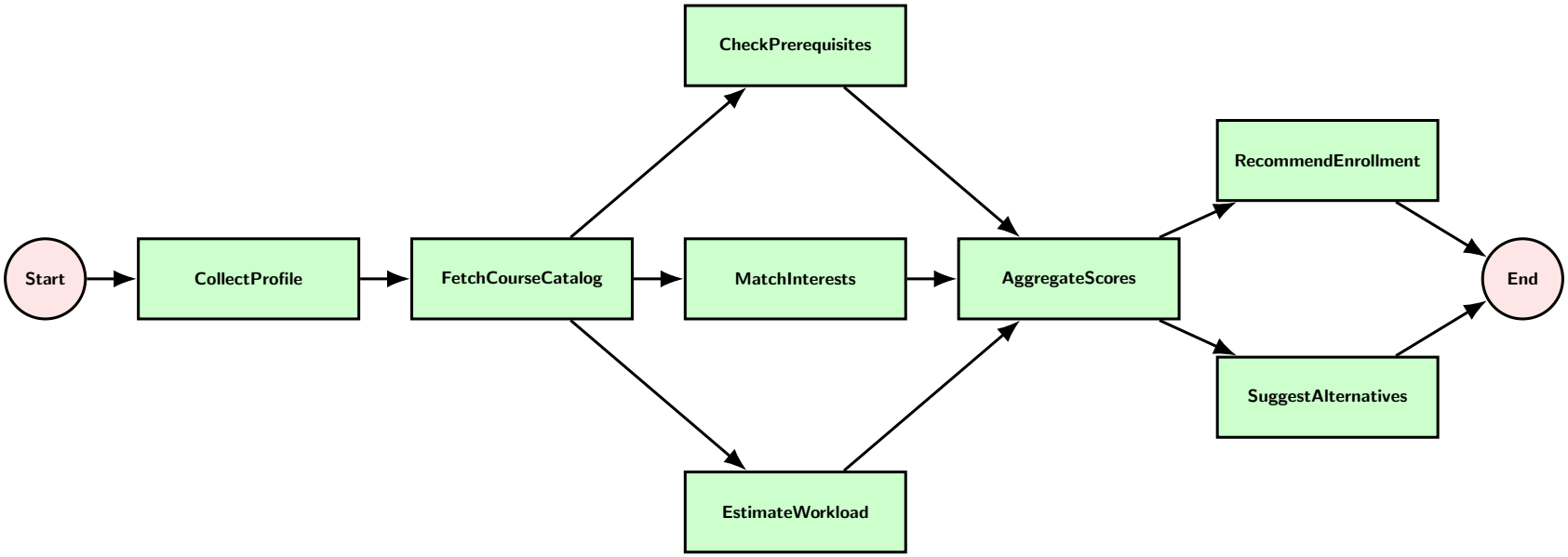
Use Case: Online Course Recommendation

Scenario: A system recommends courses to a student based on interests, prerequisites, and performance, combining multiple evaluation criteria in parallel, and then routes the student to either enrollment or guidance paths.

1. **CollectProfile**
   - System gathers student's interests, skill level, and past course history.
2. **FetchCourseCatalog**
   - Retrieves available courses from the database.
3. **EvaluateCourses (Three evaluation tasks run in parallel)**
   - **CheckPrerequisites**: Ensures student meets course prerequisites.
   - **MatchInterests**: Scores alignment with student interests.
   - **EstimateWorkload**: Evaluates course difficulty vs. student capacity.
4. **AggregateScores**
   - Combines the parallel scores into a total recommendation score.
5. **ConditionalRouting**
   - High score $\rightarrow$ **RecommendEnrollment**
   - Low score $\rightarrow$ **SuggestAlternatives**
6. **RecommendEnrollment**
   - Guides the student to enrollment for the selected course(s).
7. **SuggestAlternatives**
   - Suggests other courses, skill-building resources, or prerequisites.

Graph, Nodes, and Edges: Definitions

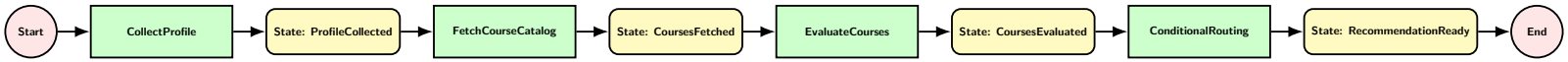
Use Case Recap: Online Course Recommendation



Graph Concepts:

- **Graph:** Entire workflow forms a graph - nodes connected by edges.
- **Node:** Each step (e.g., `CollectProfile`, `CheckPrerequisites`) is a node representing a unit of computation or action.
- **Edge:** Arrows between nodes - flow of information or conditional transitions.
- Graphs can include **parallel nodes**, **loops**, and **conditional edges** for complex workflows.

LangGraph Concept: States



Key Points:

- A **State** captures the current progress of the workflow.
- Each node execution can **update the state**, e.g., after CollectProfile → `ProfileCollected`.
- Conditional logic or parallelism decisions depend on the current **state**.
- States allow LangGraph to **resume workflows, checkpoint, and monitor execution**.
- Example: `CoursesEvaluated` state enables routing to `RecommendEnrollment` or `SuggestAlternatives`.

LangGraph State: Dictionary Representation

Initial State:

```
state = {
    "profile_collected": False,
    "courses_fetched": False,
    "courses_evaluated": False,
    "recommendation_ready": False,
    "selected_courses": []
}
```

After CollectProfile + FetchCourseCatalog:

```
state = {
    "profile_collected": True,
    "courses_fetched": True,
    "courses_evaluated": False,
    "recommendation_ready": False,
    "selected_courses": []
}
```

LangGraph State Update: Parallel Execution

Parallel Nodes:

- `CheckPrerequisites` → verifies eligibility
- `MatchInterests` → scores interest alignment
- `EstimateWorkload` → estimates course load suitability

State After Parallel Evaluation:

```
state = {
    "profile_collected": True,
    "courses_fetched": True,
    "courses_evaluated": True,
    "recommendation_ready": False,
    "selected_courses": [
        {"course_id": 101, "score": 8.5},
        {"course_id": 205, "score": 7.2}
    ]
}
```

Explanation:

- All three nodes execute **simultaneously**.
- Each node can update one or more fields in the **state dictionary**.
- After completion, the *courses_evaluated* flag is set to **True**.
- Computed outputs (like scores or recommended courses) are aggregated in *"selected_courses"*.
- LangGraph uses this updated state to determine the next step (`ConditionalRouting`).

LangGraph: Conditional Routing & State-based Decisions

Current State After Evaluation:

```
state = {
    "profile_collected": True,
    "courses_fetched": True,
    "courses_evaluated": True,
    "recommendation_ready": False,
    "selected_courses": [
        {"course_id": 101, "score": 8.5},
        {"course_id": 205, "score": 7.2}
    ]
}
```

Routing Logic Example:

```
if max(course["score"] for course in state["selected_courses"]) >= 8:
    next_node = "RecommendEnrollment"
else:
    next_node = "SuggestAlternatives"
```

Explanation:

- LangGraph evaluates conditions based on the **current state**.
- The next node is dynamically selected according to **workflow rules**.
- Conditional routing allows handling multiple paths:
  - High-scoring courses $\rightarrow$ `RecommendEnrollment`
  - Low-scoring courses $\rightarrow$ `SuggestAlternatives`
- This ensures the workflow is **adaptive** and state-driven.

LangGraph Concept: Reducers

Definition: A **Reducer** is a special function or node in LangGraph that **aggregates outputs** from multiple parallel nodes or tasks into a single result or state update.

Why Reducers are Needed:

- Parallel nodes (e.g., evaluations) produce multiple outputs simultaneously.
- The workflow often requires a **combined result** to decide the next step.
- Reducers allow LangGraph to:
  - Merge outputs (e.g., sum, average, max, min)
  - Generate a final state or decision
  - Maintain a single source of truth in the workflow

Example: Online Course Recommendation Use Case

- Parallel nodes: `CheckPrerequisites`, `MatchInterests`, `EstimateWorkload`
- Each produces a score for a course.
- **Reducer: AggregateScores**
  - Combines the three scores into a *total_score* per course.
  - Updates state dictionary: *courses_evaluated* = *True*
  - Enables next step: *ConditionalRouting*

LangGraph: State Variable Storage Types

Initial State:

```
state = {
    "profile_collected": False,          # Boolean -> replaced
    "courses_fetched": False,             # Boolean -> replaced
    "courses_evaluated": False,           # Boolean -> replaced
    "recommendation_ready": False,        # Boolean -> replaced
    "selected_courses": [],               # List -> append
    "evaluation_logs": []                 # List -> append
}
```

After One Iteration (Parallel Evaluation + Reducer):

```
state = {
    "profile_collected": True,            # replaced
    "courses_fetched": True,              # replaced
    "courses_evaluated": True,            # replaced
    "recommendation_ready": False,        # still False
    "selected_courses": [                 # appended
        {"course_id": 101, "score": 8.5},
        {"course_id": 205, "score": 7.2}
    ],
    "evaluation_logs": [                  # appended
        {"CheckPrerequisites": "Pass"},
        {"MatchInterests": 0.85},
        {"EstimateWorkload": 0.9}
    ]
}
```

LangGraph Workflow-recap

Step 1: Node Execution

```
CollectProfile -> state["profile_collected"] = True
FetchCourses -> state["courses_fetched"] = True
```

Step 2: Parallel Evaluation

```
CheckPrerequisites -> 0.9
MatchInterests     -> 0.85
EstimateWorkload   -> 0.8
```

Step 3: Reducer Aggregation

```
Reducer: AggregateScores
state["courses_evaluated"] = True
state["selected_courses"] = [
    {"course_id":101,"score":0.85},
    {"course_id":205,"score":0.8}
]
```

Step 4: Conditional Routing

```
if max(course["score"] for course in state["selected_courses"]) >= 0.8:
    next_node = "RecommendEnrollment"
else:
    next_node = "SuggestAlternatives"
```

Step 5: Final State

```
state["recommendation_ready"] = True
```

LangGraph Execution Lifecycle

LangGraph executes a workflow in multiple structured phases. The key steps are:

1. **Graph Definition:**
   - Define the workflow as a graph of **nodes** (tasks) and **edges** (dependencies).
   - States, reducers, and conditional routing are specified here.
2. **Compilation:**
   - Verify the graph structure for correctness.
   - Prepare internal representations for execution (topological order, parallel blocks, etc.).
3. **Invocation:**
   - The workflow is triggered with an initial **state dictionary**.
   - Execution begins from the designated start node.
4. **Super Steps:**
   - Execution progresses in **super steps**, where all ready nodes execute simultaneously (parallelism) or sequentially.
5. **Message Passing and Node Execution:**
   - Nodes receive inputs from parent nodes, update the state, and pass outputs to child nodes via edges.
   - Reducers aggregate outputs when multiple nodes converge.
6. **Halting Condition:**
   - Workflow execution stops when terminal node(s) are reached or no further nodes are ready to execute.

Thank You!